

Exceptions — Խորը ուղեցույց

C++ Exception Handling ամբողջական presentation

Այս ուղեցույցը մանրամասն բացատրում է C++ exception-ները՝ try/catch/throw, stack unwinding, exception safety, RAII-ի հետ կապ, noexcept, custom exception, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է Exception-ը

Exception-ը **սխալի (error) մշակման մեխանիզմ** է: Երբ սխալ է պատահում, կոդը «throw» է անում exception, որ «catch» է լինում այլ տեղում -- սարքում է normal հոսքը ու error-ների handling-ը առանձին:

```
#include <stdexcept>

int divide(int a, int b) {
    if (b == 0)
        throw std::runtime_error("Division by zero"); // throw
    return a / b;
}

int main() {
    try {
        divide(10, 0);
    } catch (const std::runtime_error& e) { // catch
        cout << "Error: " << e.what();
    }
}
```

2. try / catch / throw

```
try {  
    // կող որ կարող է throw անել  
    throw std::runtime_error("problem");  
}  
catch (const std::runtime_error& e) {    // կոնկրետ տիպ  
    cout << e.what();  
}  
catch (const std::exception& e) {      // base տիպ (բոլոր standard)  
    cout << e.what();  
}  
catch (...) {                          // ամեն ինչ (catch-all)  
    cout << "unknown error";  
}
```

```
throw    -> exception նետ  
try      -> «պահպանված» սեկցիա  
catch    -> exception բռնում, տեսակով ընտրում (վերևից վար)
```

3. Stack Unwinding (ամենակարևորը)

Երբ throw է լինում, C++-ը «հետ է գնում» (unwind) call stack-ով մինչև գտնի համապատասխան catch. Այդ ընթացքում բոլոր local object-ների destructor-ները կանչվում են -> RAII cleanup:

```
void f() {  
    std::string s = "local";           // RAII object  
    std::vector<int> v(100);           // RAII object  
    throw std::runtime_error("x");    // throw  
    // s ու v-ի destructor-ները ԱՎՏՈՄԱՏ կանչվում (stack unwinding)  
}
```

Դրա համար RAII-ն exception-safe է. throw-ի ժամանակ resource-ները ավտոմատ ազատվում են:

4. Exception-ների հիերարխիա (std::exception)

```
std::exception (base, .what())
+-- std::logic_error
|   +-- std::invalid_argument
|   +-- std::out_of_range      (vector::at, etc.)
|   +-- std::length_error
+-- std::runtime_error
    +-- std::overflow_error
    +-- std::range_error
std::bad_alloc      (new անհաջողություն)
std::bad_cast       (dynamic_cast reference)
```

```
try {
    std::vector<int> v(3);
    v.at(10);                // throw std::out_of_range
} catch (const std::out_of_range& e) {
    cout << e.what();
}
```

5. Custom Exception

```
class MyException : public std::exception {
    std::string msg;
public:
    MyException(const std::string& m) : msg(m) {}
    const char* what() const noexcept override { // override what()
        return msg.c_str();
    }
};

throw MyException("custom error");
```

Սովորաբար՝ ժառանգել `std::exception` -ից, override անել `what()` :

6. Exception Safety Guarantees

Ֆունկցիաների exception safety-ի 4 մակարդակ:

1. No-throw (noexcept) -> երբեք չի throw (ամենախիստ)
2. Strong -> throw-ի դեպքում վիճակը չի փոխվում (rollback, «all or nothing»)
3. Basic -> throw-ի դեպքում invariant պահվում (չկա leak), բայց վիճակ կարող փոխվել
4. No guarantee -> կարող է leak / corruption (ՎԱՏ)

RAII-ն սովորաբար ապահովում է առնվազն **basic** guarantee (չկա leak):

7. noexcept

```
void safe() noexcept {           // խոստանում է երբեք չի throw
    // ...
}
```

```
void safe() noexcept {
    throw 1;    // noexcept function throw անում -> std::terminate (crash)
}
```

noexcept -ը. (1) compiler optimization, (2) move constructor-ները noexcept լինելու դեպքում STL-ն օգտագործում է move (ոչ copy): Եթե noexcept function throw է անում ->

std::terminate:

8. Կարևոր կանոններ

- catch by reference (const std::exception&), ՈՉ by value (slicing)
- Destructor-ից չthrow (throw dtor-ից -> std::terminate)
- RAII օգտագործիր -> exception-safe cleanup
- catch կոնկրետից -> general (out_of_range առաջ, exception հետո)
- Մի օգտագործիր exception normal control flow-ի համար (թանկ է)

catch by reference (ոչ value)

```
catch (std::exception e)           // Սկզբ -- slicing, copy
catch (const std::exception& e)    // Ճիշտ -- reference, polymorphic
```

9. Հարցազրույցի հարցեր և պատասխաններ

Exception ինչ է:

Error-ների մշակման մեխանիզմ; throw/try/catch; normal հոսքը ու error handling առանձին:

Stack unwinding ինչ է:

throw-ի ժամանակ C++-ը հետ է գնում call stack-ով catch գտնելու; այդ ընթացքում local object-ների destructor-ներ կանչվում (RAII cleanup):

Ինչո՞ւ catch by reference:

By value -> slicing (derived exception-ը կտրվում) + copy. Reference -> polymorphic, չկա slicing:

Destructor-ից կարելի է throw անել:

Ոչ -- stack unwinding-ի ժամանակ երկրորդ throw -> std::terminate. dtor պետք է noexcept:

noexcept ինչ է:

Խոստում որ function չի throw; compiler optimization + STL move օգտագործում. Throw անելիս -> std::terminate:

Exception safety guarantees:

No-throw, strong (rollback), basic (չկա leak), no guarantee. RAII -> առնվազն basic:

std::exception հիերարխիա:

Base std::exception (.what()); logic_error (invalid_argument, out_of_range), runtime_error (overflow), bad_alloc, bad_cast:

RAII-ի հետ կապը:

RAII ապահովում է exception-safe cleanup (stack unwinding -> dtor -> resource ազատում):

Ամփոփում (Cheat Sheet)

EXCEPTION = error handling: throw / try / catch

```
throw std::runtime_error("msg");  
try { ... } catch (const std::exception& e) { e.what(); } catch(...) {}
```

STACK UNWINDING: throw -> հետ call stack-ով -> local dtor-ներ կանչվում (RAII cleanup)

HIERARCHY: std::exception -> logic_error (out_of_range) / runtime_error / bad_alloc

SAFETY: no-throw(noexcept) > strong(rollback) > basic(no leak) > none

ԿԱՆՈՆՆԵՐ:

```
catch by REFERENCE (const&) ոչ value (slicing)  
dtor-ից ՉTHROW (std::terminate)  
noexcept -> չի throw (ալլապես terminate) + optimization  
RAII -> exception-safe cleanup  
custom: ժառանգել std::exception, override what()
```